

Módulo 7: HTTP y consumo de APIs (Smocker)

Contenidos clave

requests, aiohttp y httpx (http2): Las tres son librerías para hacer peticiones HTTP, pero tienen diferencias.

- requests: Es una de las más conocidas para hacer solicitudes HTTP de forma sencilla. Hace una petición y recibe una respuesta. Se usa para peticiones HTTP síncronas y sencillas. requests -> espera respuesta -> continúa.
- aiohttp: está diseñado para trabajar con programación asíncrona utilizando async/await. aiohttp -> espera respuesta -> aprovecha ese tiempo para realizar otras tareas. Es útil cuando se tienen muchas operaciones de I/O, cómo múltiples operaciones a APIs.
- httpx y HTTP/2: http es otra librería moderna para HTTP. Puede trabajar de forma síncrona y asíncrona. Puede soportar HTTP/2, el cuál es una versión más moderna del protocolo HTTP que permite multiplexar varias solicitudes sobre una misma conexión.

Timeouts, reintentos, manejo de errores:

- timeout: establece cuánto tiempo estás dispuesto a esperar una respuesta. Si no se recibe una respuesta, se genera una excepción.
- reintentos: en aplicaciones reales se utiliza un backoff.
- Manejo de errores HTTP:
Códigos importantes: 2xx -> éxito, 3xx -> redirección, 4xx -> error relacionado con la solicitud/cliente, 5xx -> error del servidor.

Streaming de respuestas y uso eficiente de memoria: Con streaming se recibe el archivo por partes, ya que cargar todo el contenido en memoria no es eficiente. La memoria utilizada se mantiene mucho más controlada: Archivo 1 GB -> chunk pequeño -> disco -> siguiente chunk -> disco. Es especialmente útil para archivos grandes; imágenes, videos, backups, CSV grandes y respuestas grandes de APIs.

Objetivos

- Construir clientes HTTP robustos.

La clase ResilientHTTPClient usa http para realizar peticiones HTTP

```
75     with (  
76         httpx.Client(timeout=self.timeout, follow_redirects=True) as client  
77         client.stream("GET", url) as response,  
78     ):
```

Configura timeouts evitando que la aplicación se quede esperando indefinidamente:

```
43     self.timeout = httpx.Timeout(  
44         timeout_seconds,  
45         connect=5.0,  
46         read=timeout_seconds,  
47     )
```

Maneja errores de red y timeout:

```
100         except (httpx.TimeoutException, httpx.NetworkError) as err:
101             logger.warning(
102                 "Fallo de red o timeout [%s]: %s", type(err).__name__,
103             )
```

Maneja errores HTTP: Dependiendo si el servidor responde con un 4xx o 5xx, se genera un `HTTPStatusError`.

```
79         response.raise_for_status()

104     except httpx.HTTPStatusError as err:
105         logger.error(
106             "Error de respuesta HTTP %d desde el servidor: %s",
107             err.response.status_code,
108             err,
109         )
```

Tiene reintentos:

```
63         while attempt <= self.max_retries:
```

Tiene backoff entre los intentos: lo que hace que el cliente sea más resistente ante fallos temporales:

```
115         sleep_time = self.backoff_factor**attempt
116         logger.info("Reintentando en %.1f segundos...", sleep_time)
117         time.sleep(sleep_time)
```

- Gestionar resiliencia y errores.

Timeout -> `httpx.TimeoutException`

Error de red -> `httpx.NetworkError`

Error HTTP -> `HTTPStatusError`

Error 4xx -> No reintentar

Fallo temporal -> Reintenta

Fallos repetidos -> `RuntimeError`

Espera entre reintentos -> Backoff

Archivo grande -> Streaming: utiliza `client.stream("GET", url)` e itera sobre los datos en bloques de 8 KB (`iter_bytes(chunk_size=8192)`). Esto permite descargar archivos de cualquier tamaño sin impactar el consumo de memoria.

```
49     def download_file_stream(  
50         self,  
51         url: str,  
52         destination: Path,  
53         chunk_size: int = 8192,  
54     ) -> DownloadResult:  
55         """Descarga un recurso mediante streaming directo a disco,  
56  
57         aplicando reintentos con backoff exponencial.  
58         """  
59         destination.parent.mkdir(parents=True, exist_ok=True)  
60         start_time = time.perf_counter()  
61         attempt = 0
```

```
75         with (  
76             httpx.Client(timeout=self.timeout, follow_redirects=True) as client,  
77             client.stream("GET", url) as response,  
78         ):  
79             response.raise_for_status()  
80  
81             with destination.open("wb") as file_out:  
82                 for chunk in response.iter_bytes(chunk_size=chunk_size):  
83                     file_out.write(chunk)  
84                     bytes_count += len(chunk)
```

Laboratorio

- Cliente httpx con reintentos/timeouts; descarga por streaming a disco.

La clase ResilientHTTPClient encapsula la lógica de infraestructura HTTP.

http client con retries/timeouts y stream download to disk, los datos llegan por bloques y cada bloque se escribe directamente en el archivo.

El bloque `if __name__ == "__main__":` ejecuta la prueba real descargando un buffer de bytes desde un servidor público (httpbin.org), demostrando el flujo completo en ejecución.

```
125 if __name__ == "__main__":  
126     download_dir = Path(__file__).parent / "downloads"  
127     output_filepath = download_dir / "sample_data.bin"  
128  
129     # URL pública de prueba para transmisión de bytes  
130     target_url = "https://httpbin.org/bytes/50000"  
131  
132     cliente = ResilientHTTPClient(  
133         timeout_seconds=5.0, max_retries=2, backoff_factor=1.5  
134     )  
135  
136     try:  
137         resultado = cliente.download_file_stream(  
138             url=target_url, destination=output_filepath  
139         )  
140         print(f"\nReporte de Descarga: {resultado}")  
141         # BLE001: Captura explícita de las excepciones esperadas  
142         except (httpx.HTTPError, RuntimeError) as exc:  
143             logger.error("La operación falló definitivamente: %s", exc)
```

Archivos obtenidos:

Katherine Quiryat Rosales Garduño

